

A Modular Framework for LLM-Guided Pattern Analysis

Anonymous Author(s)

ABSTRACT

Exploratory pattern analysis is difficult not simply because data are large, but because end-to-end analytical workflows require coordinated, strategic reasoning over textual and tabular data alike. In this work, we present a modular framework for LLM-assisted pattern analysis. Our framework formulates inputs to language models in the forms of triples containing (a) mathematical definitions, (b) corresponding textual interpretation, and (c) a set of example pairs gluing textual and tabular evidence. Based on these, we map users from partially specified pattern components to target components, thus enabling pattern recognition, formulation, as well as instance discovery to be expressed within a unified formal representation.

We investigate whether such complementary representations of pattern knowledge, including examples, textual descriptions, formal definitions, and their combinations, improve LLM-guided exploration in an end-to-end, analyst-like workflow environment. We instantiate the approach through a suite of prompting strategies and evaluate our proof-of-concept over the USPTO dataset and its corresponding WIPO report corpus. Our results validate that modular, component-wise grounding can help make LLM reasoning more coherent in exploratory pattern analysis, while also highlighting the current limits of LLMs for the same.

PVLDB Reference Format:

Anonymous Author(s). A Modular Framework for LLM-Guided Pattern Analysis. PVLDB, 14(1): XXX-XXX, 2020.
doi:XX.XX/XXX.XX

PVLDB Artifact Availability:

The source code, data, and/or other artifacts have been made available at [URL_TO_YOUR_ARTIFACTS](#).

1 INTRODUCTION

Pattern analysis using large language models (LLMs) is challenging not only because data are large, but because meaningful patterns must often be articulated before they can be searched, compared, or validated. For non-experts, the barrier may be partially technical and partly conceptual: even when an interesting phenomenon is visible, it may not be intuitive how to formalize that observation or tie it across multiple sources of evidence. Recent advancements in language model capabilities have created new opportunities to lower this barrier by supporting more natural interaction with data. However, as the complexity associated with underlying tasks increases, purely linguistic prompting can become insufficient for

preserving the logical and mathematical discipline that complex pattern analysis necessitates.

Our problem is inherently evident in exploratory workflows that comprise a mixture of structured and unstructured data. For example, a textual report may describe an interesting pattern - for instance, a major turning point, convergence across trends, or association rules - while the corresponding evidence may live in structured counts or time series. In order to facilitate powerful human-AI workflows, LLM-powered agents must therefore do more than answer questions fluently: they must *recognize* cross-modal evidence, *formulate the mathematical logic* of a pattern, and further use that logic to *search for insights* in an integrated manner.

In this paper, we address this problem by proposing a modular framework for LLM-assisted pattern analysis. To this light, we represent a pattern in the form of pattern triple $\mathcal{P} = (\mathcal{M}, \mathcal{T}, \mathcal{E})$ as a composition of mathematical definition ($\mathcal{M}$), its textual interpretation ($\mathcal{T}$) and examples ($\mathcal{E}$). By formulating analysis as a mapping from partially known components to desired components, our proposed framework leverages the concept of component-wise grounding: the model resolves one analytical element at a time instead of collapsing the objective into a single hop. This design is intended to both reduce contextual drift and keep reasoning anchored to the available evidence.

The core focus of our work is to investigate the aforesaid design principle in contrast to claiming a general-purpose discovery engine. Our focus is to test whether complementary representations of pattern knowledge - including examples, textual descriptions, formal definitions, as well as their combinations - materially improve an LLM's ability to recognize, formalize, and operationalize patterns in an end-to-end exploratory workflow. In this paper, we investigate and test our hypothesis through a USPTO case study using structured, tabular data and a corresponding WIPO report corpus. Our key contributions include:

[leftmargin=0pt]

- We re-imagine and formalize the pattern-analysis workflows as transformations over multiple key modular pattern components.
- We propose a framework that supports component-wise grounding through mathematical definitions, textual descriptions, and example pairs.
- We perform an extensive performance evaluation study on the USPTO and WIPO datasets demonstrating how different pattern representations affect LLM-guided exploration in practice.

This work is licensed under the Creative Commons BY-NC-ND 4.0 International License. Visit <https://creativecommons.org/licenses/by-nc-nd/4.0/> to view a copy of this license. For any use beyond those covered by this license, obtain permission by emailing info@vldb.org. Copyright is held by the owner/author(s). Publication rights licensed to the VLDB Endowment.

Proceedings of the VLDB Endowment, Vol. 14, No. 1 ISSN 2150-8097.
doi:XX.XX/XXX.XX

Raghav: describe Figure 1 once doctor chengkai li approves of it.

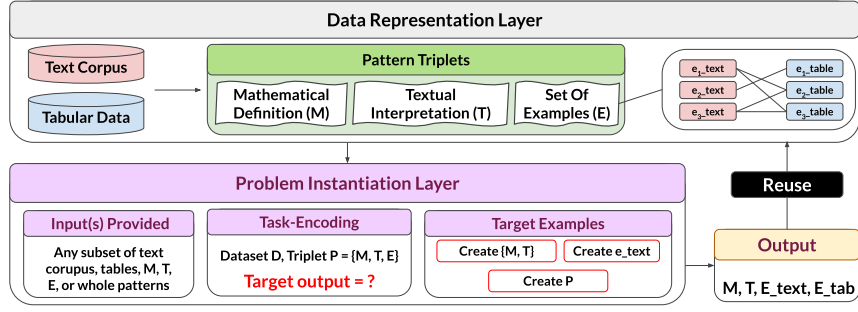


Figure 1: Overview Of Our Framework

(1) Convergence/Divergence Pattern ($P1$)			(2) Change-Point Pattern ($P2$)																																														
(1.1) Mathematical Definition ($M1$) Let $\bar{X}_A(t)$ and $\bar{X}_B(t)$ be two time series. Let $D(t) = \bar{X}_A(t) - \bar{X}_B(t)$ represent the distance between them at time t. • A Divergence Pattern exists if $D(t_{end}) > D(t_{start})$. • A Convergence Pattern exists if $D(t_{end}) < D(t_{start})$.		(1.2) Textual Description ($T1$) This pattern identifies when two different timeseries are moving closer together or pulling further apart over a period of time.	(2.1) Mathematical Definition ($M2$) Let $\bar{X}(t)$ be a time series. A change point τ is identified if there is a significant change in the trend's slope β, where: $ \beta_2 - \beta_1 \gg 0$ given β_1 is the slope for $t < \tau$ and β_2 is the slope for $t > \tau$.		(2.2) Textual Description ($T2$) This pattern identifies a specific point in time like a date, year, etc. (a "turning point") where a trend's direction or speed changes dramatically.																																												
(1.3) Instances in Report ($E1.text$) "In terms of patent growth, patent families assigned to GANs show the strongest increase over the past decade. However, patent growth of GAN patent families has slowed down and GAN patent families have increased only moderately over the last three years. In contrast, diffusion models and LLMs show much higher growth rates over the last three years..."		(1.4) Instances in Data ($E1.tab$) <table><tr><th>Grant Year</th><th>GAN Count</th><th>LLM/Diff Count</th></tr><tr><td>2018 (t_{start})</td><td>1,240</td><td>1,100</td></tr><tr><td>2019</td><td>1,350</td><td>1,450</td></tr><tr><td>2020</td><td>1,420</td><td>2,100</td></tr><tr><td>2021</td><td>1,480</td><td>4,200</td></tr><tr><td>2022</td><td>1,510</td><td>8,900</td></tr><tr><td>2023</td><td>1,550</td><td>14,300</td></tr><tr><td>2024 (t_{end})</td><td>1,590</td><td>22,100</td></tr></table>	Grant Year	GAN Count	LLM/Diff Count	2018 (t_{start})	1,240	1,100	2019	1,350	1,450	2020	1,420	2,100	2021	1,480	4,200	2022	1,510	8,900	2023	1,550	14,300	2024 (t_{end})	1,590	22,100	(2.3) Instances in Report ($E2.text$) "There has been a big surge in patent activity beginning in 2017, with an average annual growth of around 45% since then. This coincides with the introduction of transformers in 2017."		(2.4) Instances in Data ($E2.tab$) <table><tr><th colspan="2">Change Point = 2018</th></tr><tr><th>Grant Year</th><th>GenAI Count</th></tr><tr><td>2013</td><td>3,201</td></tr><tr><td>2014</td><td>3,475</td></tr><tr><td>2015</td><td>3,893</td></tr><tr><td>2016</td><td>4,686</td></tr><tr><td>2017</td><td>5,250</td></tr><tr><td>2018</td><td>5,582</td></tr><tr><td>2019</td><td>8,538</td></tr><tr><td>2020</td><td>11,399</td></tr></table>	Change Point = 2018		Grant Year	GenAI Count	2013	3,201	2014	3,475	2015	3,893	2016	4,686	2017	5,250	2018	5,582	2019	8,538	2020	11,399
Grant Year	GAN Count	LLM/Diff Count																																															
2018 (t_{start})	1,240	1,100																																															
2019	1,350	1,450																																															
2020	1,420	2,100																																															
2021	1,480	4,200																																															
2022	1,510	8,900																																															
2023	1,550	14,300																																															
2024 (t_{end})	1,590	22,100																																															
Change Point = 2018																																																	
Grant Year	GenAI Count																																																
2013	3,201																																																
2014	3,475																																																
2015	3,893																																																
2016	4,686																																																
2017	5,250																																																
2018	5,582																																																
2019	8,538																																																
2020	11,399																																																

Figure 2: Pattern cards illustrating the components of a pattern. Each card contains a mathematical definition, a textual description, and example pairs consisting of textual and tabular examples.

2 RELATED WORKS

In the context of relevant literature, this Section highlights prior efforts that focus on (a) fine-tuning task instructions, (b) in-context example selection for downstream performance optimization, and (c) benchmarks that expose the need for *explicit* decomposition.

2.1 Instruction Tuning and Task Descriptions

[18] early demonstration that models can generalize to unseen tasks when natural-language task descriptions are treated as first-class supervision

[10] showed that diverse human-written instructions can induce broad transfer across tasks

[17] established instruction tuning as a scalable recipe for zero-shot generalization, making instruction format a central design variable rather than an afterthought

[12] cast many tasks into a prompted text-to-text format and showed that multitask prompting can improve cross-task generalization without task-specific architectures

[16] expanded the instruction-tuning paradigm to a large and heterogeneous benchmark, providing a strong empirical basis for studying instruction robustness

[15] showed that instruction-following behavior can be bootstrapped from model-generated tasks and responses, underscoring the importance of instruction diversity

[7] inverted the problem by asking models to infer task descriptions from examples, which is directly relevant to our concern with representational abstractions

2.2 In-Context Learning and Example Selection

[1] made few-shot prompting a mainstream paradigm by showing that large language models can adapt to tasks from examples alone

[8] demonstrated that example order can strongly affect performance, highlighting the fragility of naive demonstration design

[9] argued that label-space and input-format regularities explain more of in-context learning than previously assumed

[11] treated prompt selection as a retrieval problem, suggesting principled ways to choose representative examples

[2] connected in-context behavior to the statistical structure of pretraining data

[4] offered a controlled theoretical perspective on the algorithmic limits of in-context learners

[20] provided a mechanistic explanation of in-context learning, which is useful when interpreting why some pattern representations transfer better than others

2.3 Benchmarks for Decomposed and Multi-Step Reasoning

[21] made multi-hop reasoning and supporting-fact identification central to question answering evaluation

[6] constructed questions with explicit reasoning-step coverage, reducing shortcuts and exposing genuine multi-hop difficulty

[3] focused on incomplete-information reading comprehension, where answering requires seeking additional context beyond the initial passage

[19] benchmarked question decomposition itself, treating the generation of subquestions as a core capability

[5] targeted questions whose reasoning strategy is implicit rather than stated, an especially relevant challenge for open-ended analytical discovery

[13] required reasoning across text, tables, and images, broadening decomposition beyond single-modality evidence paths

[14] systematically composed single-hop questions into genuine multihop problems, offering a clean benchmark for connected reasoning

3 FRAMEWORK

The goal of this framework is to formulate analytical tasks by modularizing *patterns* and *data* into components. These components act as building blocks that allow analytical problems to be expressed in a structured format to the computational tools.

Instead of defining broad research goals, the framework encourages users to express analytical tasks as mappings between patterns as **Inputs** and **Outputs**. Users take the information currently available, such as mathematical definitions of multiple patterns, examples of a single pattern, etc., and define it as the input to a tool. They then specify a desired output, which may represent new knowledge that is not currently available to them. The transformation from input to output is performed by the LLM. The input-output formulation allows users to clearly state what information is given to the tool and what result they expect in return.

3.1 Pattern and Data Representation

The framework operates on patterns as primary objects of analysis, with data providing the source from which patterns are identified and the context in which they are grounded.

This framework works with two types of data: **tabular data** ($D.tab$) and **textual data** ($D.text$). Tabular data refers to structured datasets such as spreadsheets or relational tables. Textual data refers to information expressed in natural language, such as patent landscape reports, corporate reports, or technical documents. For the purpose of this study, tabular data corresponds to a **single unified table**, and reports correspond to a **set of textual reports**.

A Pattern is defined as a recurring structural behavior found in data. For instance, in the context of multiple time-series datasets, one can observe periods where the gap between series either closes (convergence) or widens (divergence). The behavior portrayed by data during these periods can be called a convergence or divergence pattern. Similarly, all such observable structural behaviors, where data follows a recognizable and repeatable logic, we specify them as patterns.

To facilitate analysis, a pattern is modularized into these components: a Mathematical Definition (M), a Textual Description (T), and a set of Examples (E).

Examples (E) is a set comprised of pairs. Pairs are denoted by e , making $E = \{e_1, e_2, \dots\}$. Each pair (e) is composed of two parts: a **Tabular Example** ($E.tab$) and a **Textual Example** ($E.text$).

Textual Example of a pattern is an observed instance that is found in text data, such as reports. A textual example for the convergence/divergence pattern is shown in Figure 2 ($E1.text$). The textual example ($E1.text$) for the convergence/divergence pattern describes two time series, the patent count for the GAN family and the patent count for LLM/diffusion models. It notes that one is growing much faster than the other.

Corresponding to the textual example ($E.text$), there exists a tabular example ($E.tab$) found in tabular data. For the convergence/divergence pattern, $E1.tab$ is the corresponding tabular example for the textual example $E1.text$

Since, these two correspond to the same instance of a pattern in different modalities, together, ($E1.text, E1.tab$) form a pair. Although examples are represented as pairs, textual and tabular examples may also exist independently. In practice, only one form of example may be available for a pattern. The framework therefore allows analytical tasks to be formulated using either $E.text$ or $E.tab$ alone. The pair representation simply preserves the correspondence between the two forms when both are available.

The behavior illustrated by the examples can be summarized using a **textual description** (T). A textual description explains the common behavior shared by the examples using natural language. In the pattern card, the behavior observed in the example pair ($E1.text, E1.tab$) is described by the textual description T_1 in Figure 2. Unlike examples which is a set, a pattern has a single textual description that summarizes its behavior.

The pattern is also specified using a **mathematical definition** (M). The mathematical definition provides a formal rule that determines when the pattern occurs. In the convergence/divergence pattern card, the mathematical definition labeled (M_1) in Figure 2 formalizes the same behavior described in the textual description (T_1) and illustrated by the example pair ($E1.text, E1.tab$).

3.2 Problem Formulation

A problem in this framework is defined as a mapping from a set of patterns to a target pattern, supported by specific data sources. To represent these tasks, we use a notation system inspired by logic programming and regular expressions. This notation describes the information available for each pattern, and the available data

Provide URLs to report and data.

improve figure resolution.

Table 1: Representative problem formulations and signatures grounded in the USPTO study.

ID	Formulation	Description	Definition
1	$D : (_, D_{tabular})$ $P_1 : (?, T_1, _)$	Utilizing only T_1 from the convergence/divergence pattern, formulate M_1 that is suitable for $D_{tabular}$	Utilize text description of a pattern to formulate its mathematical definition, that is suitable for the provided tabular dataset
2	$D : (D.text, _)$ $P_1 : (M_1, T_1, \{(?, _)\})$	Utilizing only M_1 and T_1 from the convergence/divergence pattern, we find $E1.text$ within $D.text$ for the same.	Utilize the Mathematical definition and Text Description of a pattern to find text examples for the same pattern in reports.
3	$D : (_, D_{tabular})$ $P_1 : (_, _, \{(E1.text, ?)\})$	Utilizing only $E1.text$ from the convergence/divergence pattern, we find $E1_{tabular}$ within $D_{tabular}$ for the same.	Utilizing an observed textual example of a pattern, locate the corresponding tabular example within tabular data
4	$D : (D.text, D_{tabular})$ $P_1 : (M_1, T_1, _)$ $P_2 : (?, ?, _)$	Utilize M_1 and T_1 from the convergence/divergence pattern to synthesize M_2 and T_2 for a new pattern, namely, the change-point pattern	Utilize Math definition and text description of a known pattern as reference to synthesize the math definition and text description of a new pattern that is aligned with text and tabular data
5	$D : (_, _)$ $P_1 : (M_1, T_1, _)$ $P_2 : (?, T_2, _)$	Utilize the M_1 and T_1 from the convergence/divergence pattern and T_2 from change-point pattern as reference, to synthesize M_2 for the change-point pattern	Formulate the mathematical definition of pattern by utilizing its text description along with the math definition and text description of a new pattern as reference
6	$D : (_, _)$ $P_1 : (?, ?, \{(E1.text, E1.tab)\})$	Utilizing $E1.text$, and $E1.tab$ formulate $M1$ and $T1$ for the convergence/divergence pattern.	Utilizing one example pair for a pattern, formulate its mathematical definition and text description
7	$D : (D.text, D_{tabular})$ $P_1 : (_, _, \{(E1.text, E1.tab), (?, ?)[2]\})$	Utilizing $E1.text$ and $E1.tab$ as basis, to mine other examples of convergence/divergence pattern from $D.text$ and $D_{tabular}$	Utilize a given example pair to mine two other example pairs for the same pattern from tabular data and text reports
8	$D : (D.text, D.tab)$ $P_1 : (M1, _, \{(?, ?)[2]\})$	Utilize $M1$ to find two example pairs from $D.text$ and $D.tab$ for the convergence/divergence pattern	Utilize the mathematical definition of a pattern to mine two examples from text reports and tabular data.

sources using three primary operators. The **Underscore** ($_$) acts as a wildcard for information irrelevant to the task, the **Question Mark** ($?$) identifies a variable that serves as a goal for the LLM to discover or synthesize, and the **Fixed Count** ($[n]$), borrowed from regex, sets the number of results required. If no count is specified, the operator defaults to 1. Table 1 presents several problem formulations as queries using this system.

Within the pattern triple (M, T, E) , where E is a set of pairs $(E.text, E.tab)$, any individual component may be omitted. However, for a pattern to be valid, it must have at least one active component, meaning it must contain either a known value or a variable ($?$). For instance, in Problem 4, the input pattern $P1$ is represented as $(M1, T1, _)$. Even though the set of examples is marked as irrelevant, $P1$ is valid because it provides known values for the mathematical definition and textual description. Similarly, the target pattern $P2$ in the same problem is represented as $(?, ?, _)$. While this configuration contains no known values, it is also a valid pattern because the first two components are active variables for the LLM to solve. These examples demonstrate that the only invalid configuration for a pattern is $(_, _, _)$, as it provides no information for the tool and no goal for it to reach.

In contrast, within the data pair $(D.text, D.tab)$, either source can be omitted if not needed for the query. For instance, Problem 2

utilizes only the textual data $(D.text, _)$, while Problem 5 operates without requiring external data sources $(_, _)$.

The framework always operates over a **set of patterns**. In Problem 5, the input is a set of two patterns, namely the convergence/divergence and change-point patterns. Even, in a problem, when only a single pattern is provided, such as Problem 1 and 2, it can be treated as a singleton set, ensuring a uniform representation across all problem formulations.

The structure allows the framework to support both **intra-pattern** and **inter-pattern** problem formulations. In Problem 2, the mathematical definition and textual description of the convergence/divergence pattern are used to identify textual examples of the *same pattern* from reports, illustrating an intra-pattern transformation. In contrast, Problem 4 demonstrates an inter-pattern formulation, where the components of the convergence/divergence pattern are used to synthesize the components of a *different pattern*, namely the change-point pattern.

4 METHODOLOGY

Our methodology is organized as a case study designed to investigate the framework presented in Section 3 along three complementary dimensions. Rather than positioning the framework against a

predetermined baseline or claiming a general-purpose reasoning advantage, we examine whether the framework’s modular representation (a) changes the reliability of LLM outputs when it is made available in the prompt, (b) exposes a structured landscape of problem difficulty that is interpretable through its own components, and (c) treats its three components (M , T , E) as functionally distinct carriers of pattern knowledge rather than redundant encodings.

The three research questions are investigated through two complementary experimental structures. RQ1 is addressed through a paired comparison between the framework and a non-framework prose condition on selected problem formulations from the catalog. RQ2 and RQ3 are addressed jointly through a subtractive ablation grid run entirely within the framework condition, in which input components are progressively removed while the target is held fixed. The two structures are connected: the ablation grid operationalizes a family of problem formulations within the framework’s notation, and its results inform both the difficulty landscape of RQ2 and the component-distinctness analysis of RQ3.

4.1 Case Study Setting

We instantiate our study over a bounded subset of the USPTO patent dataset and a curated corpus of WIPO patent landscape reports. From the USPTO dataset we construct pre-aggregated tabular slices scoped to the technology areas covered by the reports, at an aggregation granularity (year $\times$ field $\times$ country, with US-centric filtering) that matches the level at which the reports discuss trends. From each report we manually extract a small set of pattern instances, hand-authoring their components as gold references. The resulting case study operates over a compact and verifiable data environment where both tabular and textual evidence for each pattern can be directly inspected.

Each pattern instance is represented according to the notation introduced in Section 3. Each analytical problem is specified as a configuration of given components, absent components, and target components, following the “?” and “_” conventions defined in the framework. This notation is the sole interface through which problems are posed to the LLM in the framework-enabled conditions of our study.

4.2 Selected Problem Formulations

From the representative problem catalog introduced in Section 3 (Table 1), we select a subset of formulations that together exercise different parts of the framework. The selected formulations are chosen to (i) cover varied target components (M , T , E_{text} , E_{tab}), (ii) include both same-pattern and cross-pattern configurations, and (iii) permit the ablations required for our three research questions.

4.3 Models and Runtime

We evaluate our study across open-weight LLMs spanning small and medium scales. Specifically, we use models in the 7B-14B range for preliminary and primary results, and complement these with larger models (70B-class) where feasible for scale-behavior analysis. Inference is performed with reasoning enabled across all conditions, as the models we consider are moderately sized and reasoning is a standard mode of operation for them. Temperature is fixed across

runs to ensure comparability, with deterministic decoding where supported. All models are served locally on a GPU cluster.

4.4 Research Questions and Experimental Design

We structure our study around three research questions. RQ1 is a *comparative* question that contrasts the framework against a non-framework prose alternative. RQ2 and RQ3 are *within-framework* questions that characterize the framework’s own internal structure. They are therefore evaluated under different experimental designs: RQ1 uses a paired framework-vs-prose comparison on the selected formulations, while RQ2 and RQ3 share a single subtractive ablation grid defined in Section 4.5.

4.4.1 RQ1: Does the availability of the framework’s modular representation change the reliability of LLM outputs? We test whether exposing the LLM to the framework’s component vocabulary and notation leads to more reliable outputs than presenting the same information in unstructured prose. We hold the information content constant across conditions and vary only the representational scaffolding available to the model. In both conditions, reasoning is enabled and the model is free to organize its reasoning as it sees fit; we do not prescribe a decomposition strategy in either condition.

Framework condition. The prompt introduces the framework’s notation and component vocabulary (M , T , E , with E decomposed into (E_{text} , E_{tab}) pairs), specifies the problem in the framework’s notation, and provides the given components in their labeled form. The LLM is expected to produce the target component(s) using the framework’s vocabulary.

Non-framework condition. The prompt presents the same information in prose without component labels, notation, or structural commitments. Pattern information is woven into a natural-language narrative, and the task is described as a question or instruction in plain language.

Both conditions are templated, with the underlying pattern content held identical. We avoid leaking framework-specific structural commitments (such as the triple structure, the pair structure of examples, or the correspondence between E_{text} and E_{tab}) into the non-framework prompt. Conversely, no framework-specific instruction to decompose is added; any decomposition behavior that arises is attributed to the LLM’s own reasoning. Each selected formulation from Section 4.2 is run under both conditions, yielding a set of paired runs per pattern instance.

We measure reliability along two axes: output agreement with gold references, and output consistency across models when given the same prompt. We additionally inspect the LLM’s reasoning traces (as produced by reasoning-enabled decoding) and manually label a stratified sample for evidence of component-wise decomposition. This allows us to observe whether the framework, when made available, induces structured reasoning behavior that differs from what the LLM does spontaneously under the prose condition.

4.4.2 RQ2: Does the framework’s modular representation expose a structured landscape of problem difficulty? We test whether performance across different input–target configurations, when viewed

Select the final subset of formulations (e.g., 3–4 out of 8). Suggested set:

Specify exact models (e.g., Llama 3.1 8B, Qwen 2.5 14B, Llama 3.1 70B), inference framework (e.g., vLLM), and hardware (H100s). Specify temperature and number of runs per condition.

through the framework’s lens, reveals interpretable structure rather than arbitrary variation. If the framework’s decomposition into M , T , and E carries meaningful semantic content, we expect that (a) the difficulty of recovering different targets varies systematically, (b) this variation correlates with framework-level properties of the problem (e.g., which component is being produced, whether production requires cross-modal grounding), and (c) the observed structure is interpretable without appealing to properties external to the framework.

RQ2 is answered entirely from the ablation grid described in Section 4.5. We report accuracy organized along framework-interpretable dimensions: target component type, the amount of information available to the model, and the modality of the remaining inputs relative to the target.

4.4.3 RQ3: Are the framework’s three components functionally distinct? We test whether the components M , T , and E carry meaningfully different information by holding the target fixed and systematically varying which other components are available as input. If the three components were redundant encodings of the same underlying pattern knowledge, we would expect the choice of provided input components to have little effect on output quality. If they are functionally distinct, the LLM’s ability to produce the target should depend systematically on which components are available, with different removals producing different accuracy drops.

Like RQ2, RQ3 is answered from the ablation grid described in Section 4.5. For each fixed target, we read the contribution profile of single and paired removals. We additionally examine whether different remaining input subsets lead to qualitatively different kinds of outputs — for instance, whether outputs produced in the presence of M tend to be more structurally faithful while those produced in the presence of T tend to be more narratively coherent.

4.5 Ablation Grid for RQ2 and RQ3

RQ2 and RQ3 are both within-framework questions: they characterize how the framework behaves, independent of any comparison to a non-framework alternative. We answer both through a single subtractive ablation grid, run entirely under the framework condition. This subsection describes the grid’s structure and the task hierarchy it encodes.

Grid structure. The grid is a table with four columns — one per atomic target component (M , T , E_{text} , E_{tab}) — and nine rows indicating which components are removed from the input relative to a full-inputs baseline. Each cell corresponds to one analytical task expressible in the framework’s notation: produce the column’s target, given all components of the pattern except those specified in the row. The nine rows are (i) the full-inputs baseline with no removals, (ii) four single-component removals ($\{M\}$, $\{T\}$, $\{E_{\text{text}}\}$, $\{E_{\text{tab}}\}$), and (iii) four selected two-component removals ($\{M, T\}$, $\{E_{\text{text}}, E_{\text{tab}}\}$, $\{M, E_{\text{tab}}\}$, $\{T, E_{\text{text}}\}$). Cells in which the removed set contains the target are undefined and left blank; the remaining cells constitute the meaningful conditions under study.

Subtractive framing. We adopt a subtractive rather than additive design because it measures the *marginal contribution* of each component in the presence of the others, which we view as the most

informative regime for characterizing functional distinctness. An additive counterpart, starting from empty inputs and progressively adding components, would characterize standalone sufficiency instead; this is a complementary question that we leave to future work.

Task hierarchy. The nine rows form a shallow hierarchy organized by how much information is withheld from the model. The baseline row (no removal) corresponds to the maximum-context tasks, in which the model receives every component of the pattern other than the target. The four single-removal rows correspond to tasks in which exactly one component is withheld, and together they isolate the marginal contribution of each individual component to each target. The four double-removal rows correspond to tasks in which two related components are withheld simultaneously, and are chosen to probe specific structural claims of the framework: (i) $\{M, T\}$ withholds both definitional forms of the pattern, testing whether the mathematical and textual definitions are mutually substitutable or jointly necessary; (ii) $\{E_{\text{text}}, E_{\text{tab}}\}$ withholds the entire example set, testing whether the pair structure contributes beyond either example alone; and (iii) $\{M, E_{\text{tab}}\}$ and $\{T, E_{\text{text}}\}$ withhold one definitional and one exemplar component, testing cross-kind complementarity between definitions and examples. We deliberately do not enumerate all $\binom{4}{2} = 6$ possible double removals; the four we include are selected to correspond to framework-level claims, and the remaining two ($\{M, E_{\text{text}}\}$ and $\{T, E_{\text{tab}}\}$) do not probe a distinct structural relation beyond what the selected four already expose.

Connection to the problem catalog. Each cell of the grid instantiates a problem expressible in the notation of Section 3. For instance, the cell at (row: $\{M\}$ removed, column: E_{text}) corresponds to a problem of the form $P : (_, T, \{(? , E_{\text{tab}})\})$ with data in context — a request to produce a textual example given only the textual description and its paired tabular example. In this sense the grid is a systematic slice through the problem space of Section 3, bounded to same-pattern single-target configurations for tractability. Cross-pattern formulations, which are also part of the framework’s catalog, are not included in the grid and are left for future work or qualitative treatment.

Mapping to RQ2 and RQ3. RQ2 reads across the grid. The full-inputs baseline row gives the ceiling accuracy for each target, indicating which targets are intrinsically hardest to produce even with maximal context. The degradation pattern — how accuracy falls as components are removed — characterizes each target’s sensitivity to input composition. Together these form the difficulty landscape of the framework’s problem space.

RQ3 reads within each column. Comparing the drops produced by different single-component removals for the same target tests whether those removals carry equivalent information (components are redundant) or differentiated information (components are functionally distinct). The double-removal rows further decompose the picture: sub-additive drops indicate that the two removed components were partially substitutable, while super-additive drops indicate joint necessity.

4.6 Data and Gold Reference Construction

For each pattern instance used in our study, we construct a gold reference by manually authoring the three components (M , T , and a small E set containing at least one complete $(E_{\text{text}}, E_{\text{tab}})$ pair). The gold M is expressed in the schema vocabulary of the corresponding USPTO data slice to ensure operational validity. Gold T is written in natural language consistent with M . Gold E pairs are drawn from the data and the source WIPO report, verified by direct inspection. Because the ablation grid requires every component to be available as both a potential input and a potential target, full authoring of all four component slots per pattern instance is a prerequisite for the study rather than an additional cost.

4.7 Evaluation Protocol

For each experimental run, outputs are evaluated using a combination of automated checks, LLM-as-judge scoring, and limited manual inspection. Schema validity of produced M is checked automatically against the data slice’s column schema and type constraints. Agreement of produced components with gold references is measured through criteria-based LLM-as-judge scoring, in which a larger judge model is given both the produced and gold components along with a decomposed checklist of criteria (variable coverage, directional correctness, threshold consistency, and so on), and returns a structured judgment. We validate the judge’s scores against direct gold-reference comparisons on a subset of outputs and report the correspondence. For reasoning-trace analysis (under RQ1), we manually label a stratified sample of traces for evidence of component-wise decomposition.

5 EVALUATION

This section reports the results of the case study described in Section 4. We first present results for RQ1, which contrasts the framework and non-framework conditions on the selected problem formulations. We then present the ablation grid results and use them to answer RQ2 and RQ3.

5.1 Setup Summary

5.2 RQ1: Reliability with vs. without the Framework

For each selected problem formulation, we compare paired runs under the framework and non-framework conditions. We report per-formulation and aggregate differences in accuracy and cross-model consistency, and complement the quantitative comparison with a manual analysis of reasoning traces.

Key observations to report (to be filled in after experiments):

- Whether framework-condition outputs agree more often with gold references, in aggregate and per formulation.
- Whether framework-condition outputs are more consistent across models.
- Whether reasoning traces in the framework condition show systematically more component-wise decomposition than the non-framework condition.

- Whether any formulations show no framework benefit or a negative effect, and interpret why.

5.3 Ablation Grid Results

We present the 9×4 ablation grid introduced in Section 4.5 as the basis for both RQ2 and RQ3. Rows correspond to the removed input subset (with the first row indicating no removals, i.e., the full-inputs baseline); columns correspond to the target component. Each cell reports mean accuracy across pattern instances, with cells in which the removed set contains the target left blank.

Table 2: Subtractive ablation grid for RQ2 and RQ3. Rows indicate the components removed from the input; columns indicate the target component to be produced. Each cell reports accuracy (mean across pattern instances). Cells in which the removed set contains the target are undefined and marked “-”. The top row is the full-inputs baseline (no removals).

Removed	M	T	E_{text}	E_{tab}
(none, full inputs)	X.XX	X.XX	X.XX	X.XX
$\{M\}$	-	X.XX	X.XX	X.XX
$\{T\}$	X.XX	-	X.XX	X.XX
$\{E_{\text{text}}\}$	X.XX	X.XX	-	X.XX
$\{E_{\text{tab}}\}$	X.XX	X.XX	X.XX	-
$\{M, T\}$	-	-	X.XX	X.XX
$\{E_{\text{text}}, E_{\text{tab}}\}$	X.XX	X.XX	-	-
$\{M, E_{\text{tab}}\}$	-	X.XX	X.XX	-
$\{T, E_{\text{text}}\}$	X.XX	-	-	X.XX

5.4 RQ2: Difficulty Landscape

To answer RQ2, we examine two projections of the ablation grid. First, the full-inputs baseline row gives the ceiling accuracy for each target, indicating which targets are intrinsically hardest to produce even with maximal context. Second, the degradation profile — how accuracy falls as inputs are removed — characterizes each target’s sensitivity to input composition. Targets whose ceiling is low are hard in an absolute sense; targets whose ceiling is high but degrade sharply under removals are input-dependent.

Key observations to report:

- Which targets are hardest to recover at the ceiling (M , T , E_{text} , E_{tab})?
- Which targets are most sensitive to input removal (i.e., most input-dependent)?
- Whether cross-modal grounding (e.g., producing E_{tab} from text-only inputs) is systematically harder than within-modality production.
- Whether the observed difficulty pattern is interpretable through framework-level properties rather than ad-hoc task characteristics.

5.5 RQ3: Component Distinctiveness

To answer RQ3, we read each column of the ablation grid and compare the accuracy drops produced by different single-component removals. If the three removal rows (M -removed, T -removed, E -component-removed) yielded equal drops for a given target, we would conclude that the components are functionally redundant for producing that target. Systematically differentiated drops indicate functional distinctness: the components carry different information that the LLM uses differently depending on the target.

We further examine the two-component removal rows to probe specific complementarity relations. The $\{M, T\}$ removal tests whether the two definitional forms are mutually substitutable (sub-additive drop) or jointly necessary (super-additive drop). The $\{E_{\text{text}}, E_{\text{tab}}\}$ removal tests the analogous question for the paired example structure. The mixed-kind removals ($\{M, E_{\text{tab}}\}$ and $\{T, E_{\text{text}}\}$) test cross-kind complementarity.

Key observations to report:

- Whether single-component removals produce differentiated drops, indicating functional distinctness.
- Whether certain components are more informative for certain targets (e.g., M more critical for producing E_{tab} , T more critical for producing E_{text}).
- Whether removing $\{M, T\}$ together produces a sub-additive or super-additive drop, revealing the relationship between the two definitional forms.
- Whether removing $\{E_{\text{text}}, E_{\text{tab}}\}$ together produces a sub- or super-additive drop, informing the value of the pair structure.
- Qualitative differences in outputs conditioned on different remaining input subsets.

5.6 Qualitative Analysis

We complement the quantitative results with a small number of qualitative examples. We present side-by-side comparisons illustrating (i) a case where the framework condition produces a cleanly decomposed and correct answer while the non-framework condition produces an ungrounded or conflated one, (ii) a case where the non-framework condition spontaneously produces decomposition comparable to the framework condition, illustrating the limits of the framework’s advantage, and (iii) a case where both conditions fail, illustrating the present limits of LLM capabilities for this problem space.

LIMITATIONS

The pattern analysis framework presented in this paper seeks to benefit from human-in-the-loop methodologies (which this paper defers to future efforts). The performance of our framework is inherently reliant on the ground-data provided to the language model, which is in-turn determined by underlying pattern-extraction and text-extraction tools. It is further constrained by the reasoning capabilities of the embedded LLM, which our framework considers as black-box. Further note, that our solution is only as good as

the quality of both tabular as well as textual data provided by an end-user (garbage-in garbage-out). We leave deeper investigations into these problems to domain experts and/or to future research.

REFERENCES

- [1] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, Sandhini Agarwal, Ariel Herbert-Voss, Gretchen Krueger, Tom Henighan, Rewon Child, Aditya Ramesh, Daniel Ziegler, Jeffrey Wu, Clemens Winter, Chris Hesse, Mark Chen, Eric Sigler, Mateusz Litwin, Scott Gray, Benjamin Chess, Jack Clark, Christopher Berner, Sam McCandlish, Alec Radford, Ilya Sutskever, and Dario Amodei. 2020. Language Models are Few-Shot Learners. In *NeurIPS*, Vol. 33. 1877–1901.
- [2] Stephanie C.Y. Chan, Adam Santoro, Andrew K. Lampinen, Jane X. Wang, Aditya K. Singh, Pierre H. Richemond, James L. McClelland, and Felix Hill. 2022. Data distributional properties drive emergent in-context learning in transformers. In *NeurIPS*. 18878–18891.
- [3] James Ferguson, Matt Gardner, Hannaneh Hajishirzi, Tushar Khot, and Pradeep Dasigi. 2020. IIRC: A Dataset of Incomplete Information Reading Comprehension Questions. In *EMNLP*. 1137–1147.
- [4] Shivam Garg, Dimitris Tsipras, Percy Liang, and Gregory Valiant. 2022. What can transformers learn in-context? a case study of simple function classes. In *NeurIPS*. 30583–30598.
- [5] Mor Geva, Daniel Khashabi, Elad Segal, Tushar Khot, Dan Roth, and Jonathan Berant. 2021. Did Aristotle Use a Laptop? A Question Answering Benchmark with Implicit Reasoning Strategies. *Transactions of the Association for Computational Linguistics* 9 (2021), 346–361.
- [6] Xanh Ho, Anh-Khoa Duong Nguyen, Saku Sugawara, and Akiko Aizawa. 2020. Constructing A Multi-hop QA Dataset for Comprehensive Evaluation of Reasoning Steps. In *COLING*. 6609–6625.
- [7] Or Honovich, Uri Shaham, Samuel R. Bowman, and Omer Levy. 2023. Instruction Induction: From Few Examples to Natural Language Task Descriptions. In *ACL*. 1935–1952.
- [8] Yao Lu, Max Bartolo, Alastair Moore, Sebastian Riedel, and Pontus Stenetorp. 2022. Fantastically Ordered Prompts and Where to Find Them: Overcoming Few-Shot Prompt Order Sensitivity. In *ACL*. 8086–8098.
- [9] Sewon Min, Xixi Lyu, Ari Holtzman, Mikel Artetxe, Mike Lewis, Hannaneh Hajishirzi, and Luke Zettlemoyer. 2022. Rethinking the Role of Demonstrations: What Makes In-Context Learning Work?. In *EMNLP*. 11048–11064.
- [10] Swaroop Mishra, Daniel Khashabi, Chitta Baral, and Hannaneh Hajishirzi. 2022. Cross-Task Generalization via Natural Language Crowdsourcing Instructions. In *ACL*. 3470–3487.
- [11] Ohad Rubin, Jonathan Herzig, and Jonathan Berant. 2022. Learning To Retrieve Prompts for In-Context Learning. In *ACL: Human Language Technologies*. 2655–2671.
- [12] Victor Sanh, Albert Webson, Colin Raffel, Stephen H Bach, Lintang Sutawika, Zaid Alyafei, Antoine Chaffin, Arnaud Stiegler, Teven Le Scao, Arun Raja, et al. 2021. Multitask prompted training enables zero-shot task generalization. *arXiv preprint arXiv:2110.08207* (2021).
- [13] Alon Talmor, Ori Yoran, Amnon Catav, Dan Lahav, Yizhong Wang, Akari Asai, Gabriel Ilharco, Hannaneh Hajishirzi, and Jonathan Berant. 2021. Multi-Modal[QA]: complex question answering over text, tables and images. In *ICLR*.
- [14] Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. 2022. MuSiQue: Multihop Questions via Single-hop Question Composition. *Transactions of the Association for Computational Linguistics* 10 (2022), 539–554.
- [15] Yizhong Wang, Yeganeh Kordi, Swaroop Mishra, Alisa Liu, Noah A. Smith, Daniel Khashabi, and Hannaneh Hajishirzi. 2023. Self-Instruct: Aligning Language Models with Self-Generated Instructions. In *ACL*. 13484–13508.
- [16] Yizhong Wang, Swaroop Mishra, Pegah Alipoormolabashi, Yeganeh Kordi, Amirreza Mirzaei, Atharva Naik, Arjun Ashok, Arut Selvan Dhanasekaran, Anjana Arunkumar, David Stap, Eshaan Pathak, Giannis Karamanolakis, Haizhi Lai, Ishan Purohit, Ishani Mondal, Jacob Anderson, Kirby Kuznia, Krma Doshi, Kuntal Kumar Pal, Maitreya Patel, Mehrad Moradshahi, Mihir Parmar, Mirali Purohit, Neeraj Varshney, Phani Rohitha Kaza, Pulkit Verma, Ravsehaj Singh Puri, Rushang Karia, Savan Doshi, Shailaja Keyur Sampat, Siddhartha Mishra, Sujay Reddy A, Sumanta Patro, Tanay Dixit, and Xudong Shen. 2022. Super-NaturalInstructions: Generalization via Declarative Instructions on 1600+ NLP Tasks. In *EMNLP*. 5085–5109.
- [17] Jason Wei, Maarten Bosma, Vincent Zhao, Kelvin Guu, Adams Wei Yu, Brian Lester, Nan Du, Andrew M. Dai, and Quoc V Le. 2022. Finetuned Language Models are Zero-Shot Learners. In *ICLR*.
- [18] Orion Weller, Nicholas Lourie, Matt Gardner, and Matthew E. Peters. 2020. Learning from Task Descriptions. In *EMNLP*. 1361–1375.
- [19] Tomer Wolfson, Mor Geva, Ankit Gupta, Matt Gardner, Yoav Goldberg, Daniel Deutch, and Jonathan Berant. 2020. Break It Down: A Question Understanding Benchmark. *Transactions of the Association for Computational Linguistics* 8 (2020),

Report per-target contribution profiles (drop in accuracy for each single removal) and the sub/super-additivity analysis for the two-component removals. A grouped bar chart or small multi-panel figure works well.

183–198.

- [20] Sang Michael Xie, Aditi Raghunathan, Percy Liang, and Tengyu Ma. 2022. An Explanation of In-context Learning as Implicit Bayesian Inference. In *ICLR*.
- [21] Zhilin Yang, Peng Qi, Saizheng Zhang, Yoshua Bengio, William Cohen, Ruslan Salakhutdinov, and Christopher D. Manning. 2018. HotpotQA: A Dataset for Diverse, Explainable Multi-hop Question Answering. In *EMNLP*. 2369–2380.

A EXAMPLE APPENDIX

We now present the prompts that our framework uses to interact with different LLMs. **Raghav: The following is inspired by prompt templates used by Akshith Putta Reddy in his paper together with Doctors Jacob and Chengkai Li (CaseFacts: A Benchmark for Legal Fact-Checking and Precedent Retrieval). Raghav: kindly delete the negative examples prompts if needed .. i think they'll add more juice to our paper**

[One-Hop Prompting Baseline]

You are an expert in multimodal pattern analysis.

Task: Solve the following analytical problem formulation in one step.

Problem Formulation:

D_text: [TEXT REPORTS OR "NONE"]

D_tab: [TABLE OR "NONE"]

Input patterns: [PATTERN SET WITH COMPONENTS MARKED AS M, T, E_text, E_tab]

Target: [ONLY THE COMPONENTS MARKED WITH ? SHOULD BE PREDICTED]

Rules:

1. Use only the provided reports, table rows, and input pattern components.
2. Do not use outside knowledge.
3. Ignore every component marked with "_".
4. If the evidence is insufficient, return null for that target and explain why.
5. Output valid JSON only.

Output JSON:

```
{
  "reasoning": "...",
  "predicted_M": "... or null",
  "predicted_T": "... or null",
  "predicted_E_text": [...],
  "predicted_E_tab": [...],
  "support": {
    "report_span_ids": [...],
    "table_row_ids": [...]"}
```

[Missing Pattern Output Using Partial Inputs]

You are a pattern-analysis expert.

Task:

Complete only the missing pattern components marked with "?" for the target pattern.

Available evidence:

D_text: [TEXT REPORTS OR "NONE"]

D_tab: [TABLE OR "NONE"]

Observed components:

M: [MATHEMATICAL DEFINITION OR null]

T: [TEXTUAL DESCRIPTION OR null]

E_text: [TEXT EXAMPLES OR []]

E_tab: [TABULAR EXAMPLES OR []]

Rules:

1. Base your answer only on the provided evidence and observed components.
2. Do not invent hidden assumptions, thresholds, or variables that are unsupported.
3. Keep each predicted component concise and pattern-level, not case-specific.
4. Output valid JSON only.

Output JSON:

```
{
  "reasoning": "...",
  "predicted_pattern": {
    "M_text": "... or null",
    "M": {
      "pattern_type": "...",
      "variables": ["..."],
      "window": "...",
      "operators": ["..."],
      "thresholds": ["..."]
    },
    "T": "... or null"
  },
  "support": {
    "report_span_ids": ["..."],
    "table_row_ids": ["..."]
  }
}
```

[Text Matching]

You are a strict judge for pattern recognition in reports.

Task: Find textual examples in D_text that express the target pattern.

Target pattern:

M: [M OR null]

T: [T OR null]

Reports: [D_TEXT WITH IDS]

Rules:

1. Use only the provided reports.
2. Return only spans that explicitly or strongly implicitly express the target pattern.
3. Do not return near-miss or background spans.
4. Rank spans by strength of support.
5. Output valid JSON only.

Output JSON:

```
{
  "reasoning": "...",
  "predicted_E_text": [
    {
      "span_id": "...",
      "text": "...",
      "justification": "..."
    }
  ]
}
```

[Table Matching]

You are an expert in linking textual claims to tabular evidence.

Task: Given one or more textual examples of a pattern, identify the corresponding tabular example(s).

Inputs:

Target textual examples:

[E_text WITH ENTITY NAMES, TIME REFERENCES, AND SPAN IDS]

Table: [D_tab WITH ROW IDS]

Rules:

1. Use only the provided table.
2. The returned table rows must correspond to the same underlying instance as the text.
3. Prefer exact entity and time alignment when available.
4. If multiple candidate windows exist, rank them and explain the difference.
5. Output valid JSON only.

Output JSON:

```
{
  "reasoning": "...",
  "predicted_E_tab": [
    {
      "row_ids": ["..."],
      "entity_ids": ["..."],
      "time_window": ["start", "end"],
      "measure": "...",
      "justification": "..."
    }
  ]
}
```

[Finding New Patterns]

You are a pattern-analysis expert.

Task: Use the source pattern as reference and synthesize the target pattern.

Source pattern:

M1: [SOURCE M]

T1: [SOURCE T]

E1_text: [OPTIONAL SOURCE TEXT EXAMPLES]

E1_tab: [OPTIONAL SOURCE TABULAR EXAMPLES]

Target evidence:

D_text: [TEXT REPORTS]

D_tab: [TABLE]

Target clues:

M2: [M2 OR null]

T2: [T2 OR null]

E2_text: [OPTIONAL TARGET EXAMPLES]

E2_tab: [OPTIONAL TARGET EXAMPLES]

Rules:

1. Use the source pattern only as reference, not as a template to copy blindly.
2. Synthesize only the target components marked with "?".
3. Ensure the target pattern is supported by the target evidence.
4. Output valid JSON only.

Output JSON:

```
{
  "reasoning": "...",
  "predicted_target_pattern": {
    "M_text": "... or null",
    "M": {...},
    "T": "... or null"
  },
  "support": {
    "report_span_ids": [...],
    "table_row_ids": [...]"
  }
}
```

[Judge 1 LLM]

You are a strict evaluation judge.

Task: Determine whether the proposed target output is fully supported by the provided evidence.

Evidence:

D_text: [TEXT]

D_tab: [TABLE]

Input pattern components: [INPUT COMPONENTS]

Proposed output: [MODEL OUTPUT]

Rules:

1. Use only the provided evidence and input components.
2. If a claim is unsupported, incomplete, or inconsistent with the evidence, mark it ungrounded.
3. Do not reward plausible but unsupported extrapolation.
4. Evaluate each target component independently.
5. Output valid JSON only.

Output JSON:

```
{
  "reasoning": "...",
  "component_labels": {
    "M": "grounded | ungrounded | not_applicable",
    "T": "grounded | ungrounded | not_applicable",
    "E_text": "grounded | ungrounded | not_applicable",
    "E_tab": "grounded | ungrounded | not_applicable"
  },
  "overall_label": "grounded | ungrounded"
}
```

[Judge 2 LLM]

You are a strict evaluation judge.

Task: Read two candidate outputs for the same pattern or the same target objective. Decide whether they are:

- contradictory,
- redundant,
- or meaningfully different.

Candidate A: [OUTPUT A]

Candidate B: [OUTPUT B]

Evidence:

D_text: [TEXT OR NONE]

D_tab: [TABLE OR NONE]

Rules:

1. Use evidence first.
2. Mark "contradictory" if the two outputs cannot both be correct.
3. Mark "redundant" if they say the same thing with no meaningful difference.
4. Mark "different" if both could be valid and non-overlapping.
5. Output valid JSON only.

Output JSON:

```
{
  "reasoning": "...",
  "decision": "contradictory | redundant | different"
}
```


[Repair LLM]

You are an expert pattern analysis with expertise in conflict resolution.

Task: Resolve a pair of candidate outputs already flagged as contradictory or redundant.

Candidate A: [OUTPUT A]

Candidate B: [OUTPUT B]

Prior decision: [CONTRADICTION OR REDUNDANT]

Reason: [EXPLANATION FROM PREVIOUS STEP]

Evidence:

D_text: [TEXT OR NONE]

D_tab: [TABLE OR NONE]

Rules:

1. If one output is better supported, keep that one.
2. If both are partially correct, return a merged and corrected output.
3. If neither is supported, reject both.
4. Output valid JSON only.

Output JSON:

```
{
  "reasoning": "...",
  "decision": "keep_A | keep_B | merge | reject_both",
  "merged_output": "... or null"
}
```

[Negative LLM Generator]

You are an expert in pattern analysis.

Task:

Create a hard negative for the target pattern. The negative should be plausible and highly similar to a true instance, but it must fail one decisive condition of the pattern.

True pattern:

M: [GOLD M]

T: [GOLD T]

Positive example:

E_text: [TRUE TEXT EXAMPLE]

E_tab: [TRUE TABULAR EXAMPLE]

Rules:

1. Preserve most surface features of the positive example.
2. Change exactly one decisive condition (e.g., wrong turning point, wrong trend direction, wrong entity pair, wrong time window).
3. Explain why the negative is close but invalid.
4. Output valid JSON only.

Output JSON:

```
{
  "reasoning": "...",
  "negative_E_text": "... or null",
  "negative_E_tab": "... or null",
  "failed_condition": "..."
}
```

You are a compiler for pattern definitions.

Task:

Convert the following free-form mathematical definition into the required normalized DSL.
Do not change its meaning. If the meaning is ambiguous, return "cannot_normalize".

Input: [FREE-FORM M]

Required M fields:

pattern_type, variables, window, operators, thresholds

Output JSON:

```
{
  "status": "normalized | cannot_normalize",
  "M_": {
    "pattern_type": "...",
    "variables": ["..."],
    "window": "...",
    "operators": ["..."],
    "thresholds": ["..."]
  },
  "notes": "..."
}
```